

Python Tip: `lru_cache`

Want to speed up your code by caching function results? Use `functools.lru_cache` to store results of expensive function calls and reuse them.

Basic Example of a Slow Function

Here is a slow recursive function for Fibonacci numbers. Without caching, it recalculates the same values many times.

```
def slow_fibonacci(n):  
    print(f"- Calculating Fibonacci({n})")  
    if n <= 1:  
        return n  
    return slow_fibonacci(n - 1) + slow_fibonacci(n - 2)  
  
x = slow_fibonacci(3)  
print(f"Final result = {x}")
```

```
- Calculating Fibonacci(3)  
- Calculating Fibonacci(2)  
- Calculating Fibonacci(1)  
- Calculating Fibonacci(0)  
- Calculating Fibonacci(1)  
Final result = 2
```

Using lru_cache

You can speed this up by adding the `@lru_cache` decorator. It caches results and returns them instantly when the function is called with the same arguments.

```
from functools import lru_cache

@lru_cache(maxsize=None) # No limit on cache size
def fast_fibonacci(n):
    print(f"- Calculating Fibonacci({n})")
    if n <= 1:
        return n
    return fast_fibonacci(n - 1) + fast_fibonacci(n - 2)

x = fast_fibonacci(3)
print(f"Final result = {x}")
```

```
- Calculating Fibonacci(3)
- Calculating Fibonacci(2)
- Calculating Fibonacci(1)
- Calculating Fibonacci(0)
Final result = 2
```

Cache Management

`lru_cache` stores results in memory, not on disk. Be careful when caching values that consume a lot of memory.

You can clear the cache manually and inspect cache stats like hits, misses, and current size.

```
fast_fibonacci.cache_clear() # Clears the cache  
print(fast_fibonacci.cache_info())
```

```
CacheInfo(hits=0, misses=0, maxsize=None, currsize=0)
```


Wrap-Up

Now you can use `lru_cache` to speed up expensive function calls by reducing redundant computations.

Follow me for more tips.

Shep Bryan IV